

**PRACTICE EXERCISES OF THE MICROPROCESSORS
& MICROCONTROLLERS**

Instructor: The Tung Than

Student's name: Gia Hung Nguyen

Student code: 24520603

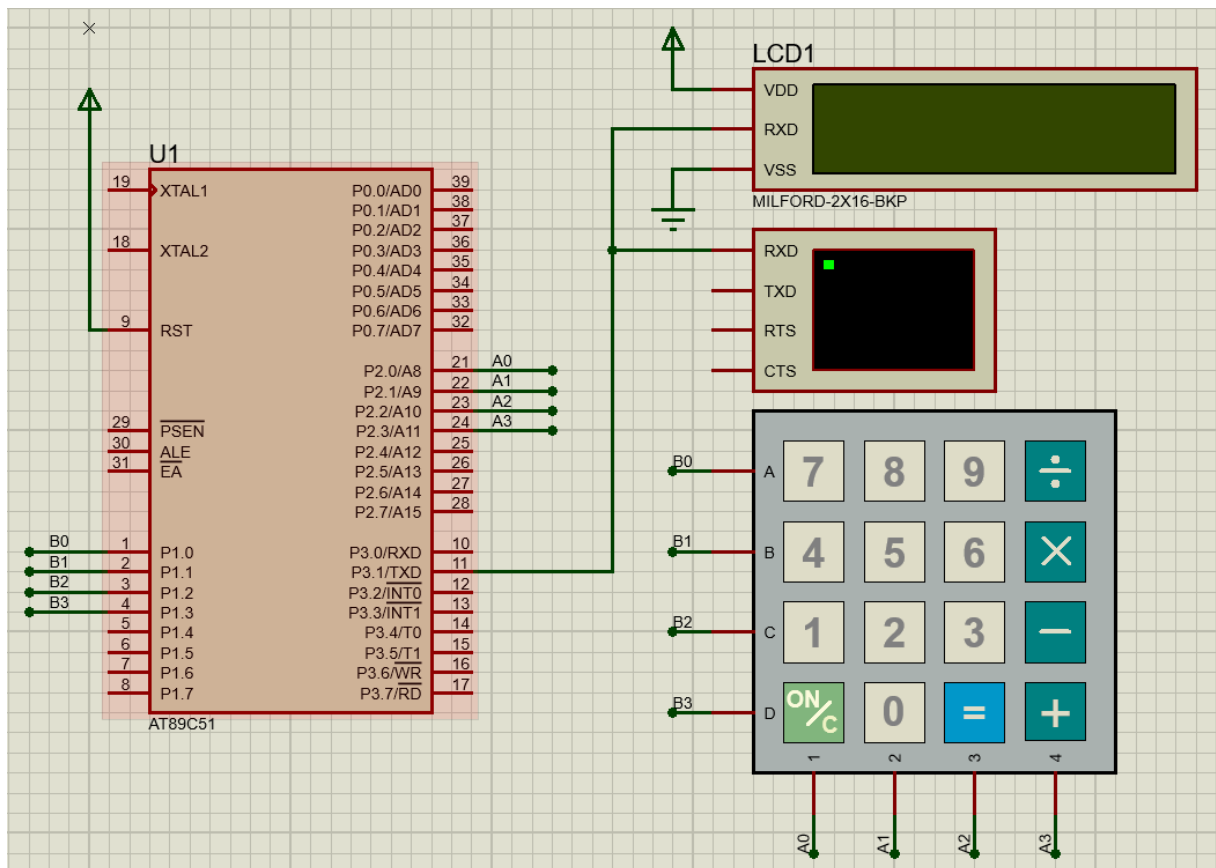
**PRACTICE REPORT NO 4
USING UART**

- I. Thiết kế phần cứng**
- II. Xây dựng chương trình Assembly**
- III. Kết quả mô phỏng**

I. Thiết kế phần cứng

Mô phỏng được thực hiện trên phần mềm Proteus với các thành phần chính để xây dựng hệ thống máy tính bỏ túi giao tiếp qua UART:

- **Khối điều khiển trung tâm: AT89C51.**
- **Keypad 4x4:** Được thiết kế để nhập các chữ số từ 0-9 và các phép toán (+, -, x, /, =, C). Bàn phím được cấu hình quét theo ma trận:
 - Các hàng được kết nối với **Port 1** (P1.0 - P1.3).
 - Các cột được kết nối với **Port 2** (P2.0 - P2.3).
- **LCD:** Giao tiếp nối tiếp (UART) thông qua chân **TXD** (P3.1). Dữ liệu ASCII được truyền với tốc độ Baudrate 9600 bps.



Hình 1. Sơ đồ mạch Calculator giao tiếp UART với Keypad 4x4 trên Proteus

II. Xây dựng chương trình Assembly

ASSEMBLY	GIẢI THÍCH
<pre> ORG 0000H LJMP START ROW EQU P1 COL EQU P2 X EQU R0 Y EQU R1 PHEP_TINH EQU R2 FLAG EQU R3 SO_DU EQU R4 ORG 0100H START: MOV TMOD, #20H MOV TH1, #0FDH MOV SCON, #50H SETB TR1 MOV X, #0 MOV Y, #0 MOV FLAG, #0 MOV PHEP_TINH, #0 MOV SO_DU, #0 </pre>	Định nghĩa biến & Khởi tạo hệ thống: Đặt tên cho các thanh ghi và các port để dễ quản lý. Cấu hình UART (9600 bps): Sử dụng Timer 1 ở Chế độ 2 (TMOD = 20H - 8-bit tự nạp lại). Nạp giá trị FDH cho TH1 để thiết lập tốc độ truyền 9600 baud (với thạch anh 11.0592MHz). SCON = 50H: UART Mode 1 (8-bit data). Bật TR1 để Timer chạy. Reset các biến tính toán về 0 trước khi bắt đầu.

ASSEMBLY	GIẢI THÍCH
<pre> KEYPAD: MOV ROW, #00H WAIT_PRESS: MOV A, COL ANL A, #0FH CJNE A, #0FH, CHECK_PRESS SJMP WAIT_PRESS CHECK_PRESS: LCALL DELAY MOV A, COL ANL A, #0FH CJNE A, #0FH, TIM_ROW SJMP KEYPAD TIM_ROW: MOV ROW, #0FEH MOV A, COL ANL A, #0FH CJNE A, #0FH, QUET_R1 MOV ROW, #0FDH MOV A, COL ANL A, #0FH CJNE A, #0FH, QUET_R2 MOV ROW, #0FBH MOV A, COL ANL A, #0FH CJNE A, #0FH, QUET_R3 MOV ROW, #0F7H MOV A, COL ANL A, #0FH CJNE A, #0FH, QUET_R4 LJMP WAIT_PRESS </pre>	Quét ma trận phím: Đầu tiên kéo tất cả các hàng xuống mức 0, rồi liên tục đọc tín hiệu từ các cột. Lệnh <code>ANL A, #0FH</code> che 4 bit cao, chỉ quan tâm trạng thái 4 bit thấp (từ P2.0-P2.3). Nếu khác (0FH) tức là có phím được nhấn. Chống dội phím: Gọi hàm <code>DELAY</code> và kiểm tra lại trạng thái. Nếu vẫn được nhấn thì tiến hành dò tìm hàng. Thuật toán dò hàng thực hiện bằng cách cấp mức logic 0 tuần tự cho từng hàng (0FEH, 0FDH, 0FBH, 0F7H) và kiểm tra lại cột để xác định tọa độ phím giao nhau. Nếu không tìm thấy ở hàng nào thì quay lại chờ phím mới.

ASSEMBLY	GIẢI THÍCH
<pre> QUET_R1: MOV DPTR, #ROW_1 SJMP TIM_COL QUET_R2: MOV DPTR, #ROW_2 SJMP TIM_COL QUET_R3: MOV DPTR, #ROW_3 SJMP TIM_COL QUET_R4: MOV DPTR, #ROW_4 SJMP TIM_COL TIM_COL: RRC A JNC NEXT INC DPTR SJMP TIM_COL NEXT: MOV A, #0 MOVC A, @A+DPTR LCALL TINH WAIT_RELEASE: MOV ROW, #00H MOV A, COL ANL A, #0FH CJNE A, #0FH, WAIT_RELEASE LJMP KEYPAD </pre>	Đọc mã ASCII & Xử lý nhả phím: Tùy vào hàng được phát hiện, con trỏ DPTR sẽ được trỏ tới mảng dữ liệu ASCII tương ứng lưu trong ROM (ROW_1, ROW_2...). Vòng lặp TIM_COL sử dụng lệnh RRC A để dò xem bit 0 nằm ở cột nào. Mỗi lần quay lệch, tăng DPTR lên 1 byte. Khi C=0 (tìm thấy phím), nhảy tới NEXT. Lệnh MOVC trích xuất ký tự ASCII đã tra cứu đưa vào thanh ghi A và gọi hàm TINH để tính toán. WAIT_RELEASE: Chờ người dùng nhả nút thì mới thoát khỏi vòng lặp, tránh việc in số liên tục.

ASSEMBLY	GIẢI THÍCH
TINH: <pre> CJNE A, #'C', CHECK_EQUAL LJMP CLEAR </pre> CHECK_EQUAL: <pre> LCALL HIENTHI CJNE A, #'=', NOT_EQUAL MOV A, PHEP_TINH </pre> CJNE A, #'+' , CHECK_SUB <pre> MOV A, X ADD A, Y SJMP DONE </pre> CHECK_SUB: <pre> CJNE A, #'-' , CHECK_MUL CLR C MOV A, X SUBB A, Y JC IN_AM LJMP DONE </pre> IN_AM: <pre> MOV B, A MOV A, #'-' LCALL HIENTHI MOV A, B CPL A INC A SJMP DONE </pre>	Tính toán: Kiểm tra nút nhấn. Nếu phím là 'C', gọi hàm CLEAR xóa màn LCD. Nếu là phím '=', nạp toán tử đã lưu vào A để tính toán. Phép Cộng: Nạp giá trị biến X vào A, dùng lệnh ADD A, Y ($X + Y$). Xong nhảy tới DONE. Phép Trừ: Lệnh SUBB A, Y thực hiện $X - Y$. Xóa cờ Carry trước khi trừ để tránh mượn sai. Nếu $X < Y$, phép trừ sinh ra mượn, cờ Carry sẽ bật ($C=1$). Hàm dùng lệnh JC IN_AM nhảy đi xử lý in số âm. In số âm: Cất tạm kết quả vào B. Nạp mã ASCII '-' vào A và gửi qua UART. Lấy lại dữ liệu từ B, chuyển kết quả âm thành dương bằng cách đảo và cộng thêm 1 và in ra.

ASSEMBLY	GIẢI THÍCH
<pre> CHECK_MUL: CJNE A, #'X', CHECK_DIV MOV A, X MOV B, Y MUL AB SJMP DONE CHECK_DIV: MOV A, Y JZ IN_ERROR MOV A, X MOV B, Y DIV AB MOV SO_DU, B LCALL IN_SO MOV A, #'.' LCALL HIENTHI </pre>	Phép Nhân: Nạp X vào A, Y vào B. Lệnh MUL AB cho kết quả tích số trong A. Phép Chia: Trước khi chia, dùng lệnh JZ IN_ERROR để kiểm tra trường hợp chia cho 0. Xử lý số thập phân (Phần nguyên): Tiến hành chia X / Y. Phần nguyên nằm trong A được gọi hàm IN_SO xuất thẳng. Phần số dư (B) được cất vào biến SO_DU. Sau đó xuất dấu chấm '.'.

ASSEMBLY	GIẢI THÍCH
<pre> MOV A, SO_DU MOV B, #10 MUL AB MOV B, Y DIV AB MOV SO_DU, B LCALL IN_SO MOV A, SO_DU MOV B, #10 MUL AB MOV B, Y DIV AB CJNE A, #5, LAM_TRON INC A SJMP DONE LAM_TRON: JC DONE INC A DONE: LCALL IN_SO MOV FLAG, #0 RET </pre>	Xử lý số thập phân (Tiếp theo): Để lấy chữ số thập phân thứ 1: Nạp lại SO_DU vào A, nhân 10, rồi chia tiếp cho Y. Lưu số dư mới và xuất chữ số thập phân ra. Lặp lại thao tác trên để lấy chữ số thập phân thứ 2. Tại lần thứ 3, thay vì in ra, chương trình kiểm tra giá trị sinh ra. Nếu ≥ 5, thuật toán sẽ làm tròn lên bằng cách cộng thêm 1 (INC A) vào kết quả thập phân cuối cùng, ngược lại thì bỏ qua. Cuối cùng, nhảy đến nhãn DONE in chữ số đã được làm tròn ra ngoài và đưa cờ FLAG về 0.

ASSEMBLY	GIẢI THÍCH
NOT_EQUAL: <pre> CJNE A, #'+' , CHECK_OPT2 MOV PHEP_TINH, A MOV FLAG, #1 RET </pre> CHECK_OPT2: <pre> CJNE A, #'-' , CHECK_OPT3 MOV PHEP_TINH, A MOV FLAG, #1 RET </pre> CHECK_OPT3: <pre> CJNE A, #'X' , CHECK_OPT4 MOV PHEP_TINH, A MOV FLAG, #1 RET </pre> CHECK_OPT4: <pre> CJNE A, #'/' , LUU_SO MOV PHEP_TINH, A MOV FLAG, #1 RET </pre>	Kiểm tra phím & Lưu toán hạng: Kiểm tra ký tự vừa nhấn thuộc Phép tính hay Chữ số. Xử lý Phép toán: Kiểm tra đối chiếu với các ký tự toán học (+, -, X, /). Nếu trùng khớp, cất ký tự đó vào thanh ghi PHEP_TINH. Thiết lập cờ FLAG = 1. Cờ này thông báo cho hệ thống biết toán hạng X đã nhập xong, từ giờ trở đi số nhập vào sẽ là của toán hạng Y.
LUU_SO: <pre> MOV B, A MOV A, FLAG CJNE A, #0, NHAP_Y CLR C MOV A, B SUBB A, #'0' MOV X, A RET </pre> NHAP_Y: <pre> MOV A, B CLR C SUBB A, #'0' MOV Y, A RET </pre>	Xử lý Chữ số: Nếu tất cả phép so sánh toán tử bên trên đều sai, ký tự nhấn chắc chắn là chữ số (0-9). Để có thể tính toán được, cần phải chuyển đổi mã ASCII sang số nguyên thực bằng cách lấy mã ASCII của chữ số trừ đi mã ASCII của '0' (SUBB A, #'0'). Hệ thống sẽ kiểm tra cờ FLAG. Nếu FLAG = 0, số đã chuyển đổi được lưu vào biến X. Nếu FLAG = 1, chương trình lưu con số vào biến Y.

ASSEMBLY	GIẢI THÍCH
CLEAR: <pre> LCALL DELAY MOV A, #0FEH LCALL HIENTHI LCALL DELAY MOV A, #01H LCALL HIENTHI LJMP START </pre> IN_ERROR: <pre> MOV DPTR, #ERROR_STR LCALL IN_STR MOV FLAG, #0 RET </pre> IN_STR: <pre> CLR A MOVC A, @A+DPTR JZ KET_THUC_CHUOI LCALL HIENTHI INC DPTR SJMP IN_STR </pre> KET_THUC_CHUOI: <pre> RET </pre>	Giao tiếp với UART: CLEAR: Điều khiển LCD. Mã 0FEH kích hoạt chế độ lệnh, mã 01H thực hiện xóa toàn bộ màn hình. IN_STR: Dùng để truyền một chuỗi ký tự dài (như "Error"). Lệnh MOVC lấy từng ký tự từ trong ROM, sau đó tăng con trỏ lên để lấy chữ cái tiếp theo. Vòng lặp sẽ dừng khi ký tự được trích xuất là khoảng trắng thông qua lệnh JZ KET_THUC_CHUOI.
IN_SO: <pre> MOV B, #10 DIV AB CJNE A, #0, IN_CHUC SJMP IN_DONVI </pre> IN_CHUC: <pre> ADD A, #'0' LCALL HIENTHI </pre> IN_DONVI: <pre> MOV A, B ADD A, #'0' LCALL HIENTHI RET </pre>	IN_SO: Chuyển con số thành chuỗi ASCII để in ra. Chia cho 10 tách riêng hàng Chục (A) và Đơn vị (B). Nếu hàng chục khác 0 (Ví dụ 15) thì in cả hai số, nếu bằng 0 (Ví dụ số 7) thì bỏ qua hàng chục tránh in ra "07". Cộng dữ liệu với mã ASCII của '0' trước khi hiển thị.

ASSEMBLY	GIẢI THÍCH
<pre> HIENTHI: MOV SBUF, A HERE: JNB TI, HERE CLR TI RET DELAY: MOV R6, #192 LOOP1: MOV R5, #192 LOOP2: NOP DJNZ R5, LOOP2 DJNZ R6, LOOP1 RET ORG 300H ROW_1: DB '7','8','9','/' ROW_2: DB '4','5','6','X' ROW_3: DB '1','2','3','- ' ROW_4: DB 'C','0','=','+' ERROR_STR: DB 'Error', 0 END </pre>	

III. Kết quả mô phỏng

Link video mô phỏng nguyên lý hoạt động của mạch và File thiết kế Proteus:

<https://drive.google.com/drive/folders/1BbAsgDphqrR-BKqPKaKvB2KuHtoeUExw?usp=sharing>